

## CSE 676: Deep Learning/final project

# Forecasting Stock Market Trends with Deep Learning

Team103/Aditya rawat

## 1. Introduction

- This project aims to predict stock trends using deep learning models, such as Long-Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks.
- The dataset is obtained using the yfinance library. Key financial indicators such as Simple Moving Averages (SMA) and Relative Strength Index (RSI) are extracted for features.
- The models are trained and evaluated using PyTorch.

### 1.1 Objectives

- Collect and preprocess stock market data from Yahoo Finance and other relevant datasets.
- Implement baseline model using LSTMs for time series forecasting.
- Improve model/accuracy by implementing GRU-based models.
- Compare model performance using evaluation metrics like RMSE and MAE.
- Deploy the final model using a Flask/Streamlit web application for user interaction.

## 2. Dataset and Preprocessing

### 2.1 Data Acquisition

Stock market data is obtained using the yfinance library, which provides historical price data such as Open, High, Low, Close, Volume, and Adjusted Close values.

```
import yfinance as yf
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

### 2.2 Feature Engineering

- Extracted key financial indicators:

- 10-day Simple Moving Average (SMA\_10) - SMA\_10 is the mean closing price of a stock over the last 10 days
- 50-day Simple Moving Average (SMA\_50) - like SMA\_10 but over a longer term (50 days). It is used to identify the medium-term trend of a stock, used to confirm if a stock is in an uptrend or downtrend in the medium term.
- Relative Strength Index (RSI)- RSI is a speed and change measure of momentum movement, and it ranges from 0 to 100. RSI helps ascertain whether a stock is overbought or oversold.
- A rolling window sequence of 20 time steps is used to create input-output pairs for time-series forecasting.

```
def fetch_stock_data(tickers, start_date, end_date):
    data = {}
    for ticker in tickers:
        df = yf.download(ticker, start=start_date, end=end_date)
        df['Ticker'] = ticker

        # Technical indicators with time shifting
        df['SMA_10'] = df['Close'].rolling(10).mean().shift(1)
        df['SMA_50'] = df['Close'].rolling(50).mean().shift(1)

        # RSI
        delta = df['Close'].diff().shift(1)
        gain = delta.where(delta > 0, 0)
        loss = -delta.where(delta < 0, 0)
        avg_gain = gain.rolling(14).mean()
        avg_loss = loss.rolling(14).mean()
        rs = avg_gain / avg_loss
        df['RSI'] = 100 - (100 / (1 + rs))

        df.dropna(inplace=True)
        data[ticker] = df
    return data
```

## 2.3 Data Splitting

- The dataset is split into training(60%), validation(20%), and test sets(20%).
- PyTorch's TensorDataset and DataLoader are used to batch and load data for training.

## 3. Model Implementation

### 3.1 Neural Network Architectures

Two deep learning models are implemented:

#### 1. LSTM

```

class StockLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim=64, output_dim=1, n_layers=1):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, n_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        output, _ = self.lstm(x)
        return self.fc(output[:, -1])

```

**LSTM Layer:** This layer is used to learn long-term dependencies in the data. It's useful for time-series prediction.

- **input\_dim:** These are number of features per time step in the input sequence.
- **hidden\_dim:** These are number of hidden units in the LSTM layer.
- **n\_layers:** The number of LSTM layers in the network.

**Fully Connected Layer:** After the LSTM processes the sequence, the output is passed through a FC(Linear) layer to reduce the dimensionality to the desired output size.

- **output\_dim:** The number of outputs.
- **GRU (Gated Recurrent Unit):** It is a simple version of LSTM that reduces computational complexity while maintaining performance.

```

class StockGRU(nn.Module):
    def __init__(self, input_dim, hidden_dim=64, output_dim=1, n_layers=1):
        super().__init__()
        self.gru = nn.GRU(input_dim, hidden_dim, n_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        output, _ = self.gru(x)
        return self.fc(output[:, -1])

```

**GRU Layer:** Like the LSTM, the GRU is another RNN model used to process sequences of data. GRU is faster computationally than LSTM.

- **input\_dim:** These are number of features per time step in the input sequence.
- **hidden\_dim:** These are number of hidden units in the GRU layer.
- **n\_layers:** The number of GRU layers.

**Fully Connected Layer:** Similar to the LSTM model, the GRU output is passed through a fully connected layer to produce the final prediction.

### **3.2 Model Specifications**

- Input Features: Close Price, SMA\_10, SMA\_50, RSI
- Hidden Dimension: 64
- Number of Layers: 1
- Loss Function: Huber Loss (to handle outliers effectively)
- Optimizer: Adam (Adaptive learning rate optimization)
- Number of Epochs: 50
- Learning Rate: 0.001

## **4. Training and Evaluation**

### **4.1 Training Process**

- Models are trained using PyTorch with GPU acceleration (if available).
- Training involves backpropagation with the Adam optimizer and Huber Loss.
- Performance metrics include Mean Absolute Error (MAE) and  $R^2$  Score.
- Based on the results best model weights are saved

```

def train_model(model, train_loader, val_loader, target_scaler, ticker, epochs=50, lr=0.001):
    import os
    import torch
    from sklearn.metrics import mean_absolute_error, r2_score
    from torch import nn

    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model = model.to(device)
    criterion = nn.HuberLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    best_r2 = -float('inf')
    history = {
        'train_loss': [],
        'train_mae': [],
        'train_r2': [],
        'val_loss': [],
        'val_mae': [],
        'val_r2': []
    }

    for epoch in range(epochs):
        # Training Phase
        model.train()
        train_loss = 0
        train_preds, train_true = [], []

        for X_batch, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)

            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch.unsqueeze(1))
            loss.backward()
            optimizer.step()

            train_loss += loss.item()
            train_preds.append(outputs.detach().cpu())
            train_true.append(y_batch.unsqueeze(1).detach().cpu())

        # Training metrics
        train_preds = torch.cat(train_preds).numpy()
        train_true = torch.cat(train_true).numpy()

        train_preds_raw = target_scaler.inverse_transform(train_preds.reshape(-1, 1)).flatten()
        train_true_raw = target_scaler.inverse_transform(train_true.reshape(-1, 1)).flatten()

        train_mae = mean_absolute_error(train_true_raw, train_preds_raw)
        train_r2 = r2_score(train_true_raw, train_preds_raw)

```

```

    # Validation Phase
    model.eval()
    val_loss = 0
    val_preds, val_true = [], []

    with torch.no_grad():
        for X_val, y_val in val_loader:
            X_val, y_val = X_val.to(device), y_val.to(device)
            outputs = model(X_val)
            val_loss += criterion(outputs, y_val.unsqueeze(1)).item()

            val_preds.append(outputs.cpu())
            val_true.append(y_val.cpu())

    # Validation metrics
    val_preds = torch.cat(val_preds).numpy()
    val_true = torch.cat(val_true).numpy()

    val_preds_raw = target_scaler.inverse_transform(val_preds.reshape(-1, 1)).flatten()
    val_true_raw = target_scaler.inverse_transform(val_true.reshape(-1, 1)).flatten()

    val_mae = mean_absolute_error(val_true_raw, val_preds_raw)
    val_r2 = r2_score(val_true_raw, val_preds_raw)

    history['train_loss'].append(train_loss / len(train_loader))
    history['train_mae'].append(train_mae)
    history['train_r2'].append(train_r2)
    history['val_loss'].append(val_loss / len(val_loader))
    history['val_mae'].append(val_mae)
    history['val_r2'].append(val_r2)

    print(f"Epoch {epoch+1}/{epochs}")
    print(f"[Train] Loss: {history['train_loss'][-1]:.4f} | MAE: ${train_mae:.2f} | R²: {train_r2:.4f}")
    print(f"[Val] Loss: {history['val_loss'][-1]:.4f} | MAE: ${val_mae:.2f} | R²: {val_r2:.4f}")

    # Save best model (ticker-based)
    model_dir = os.path.join('..', 'app', 'models', ticker)
    os.makedirs(model_dir, exist_ok=True)
    model_save_path = os.path.join(model_dir, f'best_{model.__class__.__name__}.pth')

    if val_r2 > best_r2:
        best_r2 = val_r2
        torch.save(model.state_dict(), model_save_path)
        print(f"New best model saved for {ticker} (R²: {best_r2:.4f}")

    return model, history

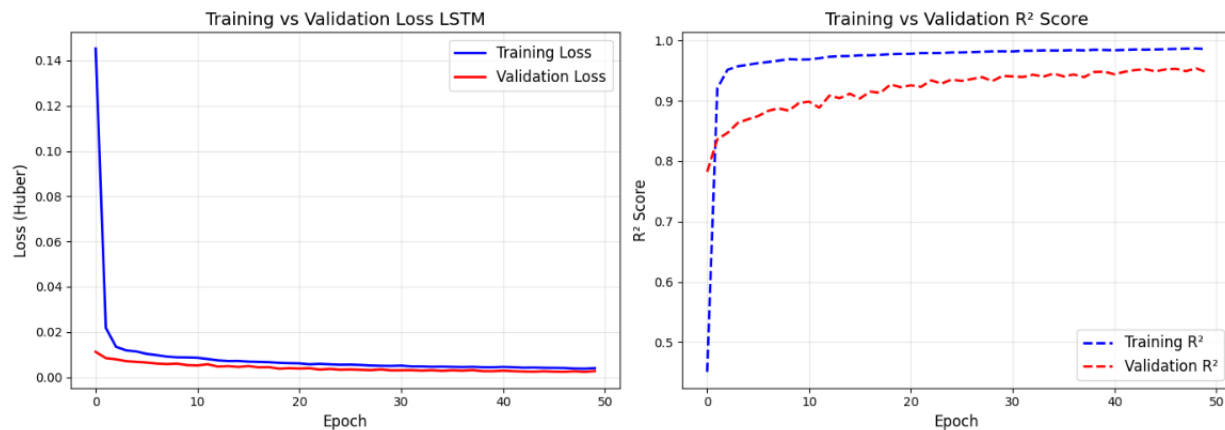
```

## 4.2 Evaluation Metrics

- **Mean Absolute Error (MAE):** Measures absolute differences between actual and predicted prices.
- **R<sup>2</sup> Score:** Indicates the proportion of variance explained by the model.
- **Loss:** measures how far off a model's predictions are from the actual target values

## 5. Training phase (i) [evaluating performance on a single stock]

- **LSTM**



### Loss

-> The training loss decreases in the initial epochs and becomes constant later, indicating good learning.

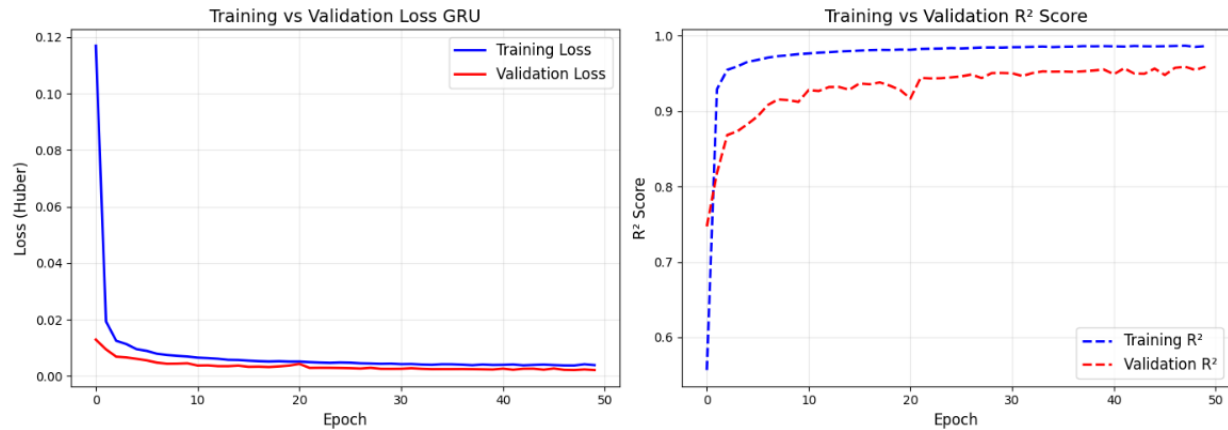
-> The validation loss follows the same trend but at a larger value, which indicates that the model is generalizing well and not overfitting.

### R<sup>2</sup> Score

-> The R<sup>2</sup> of the training rises very fast above 0.95, which reflects very good model fit with the training data. The R<sup>2</sup> of the validation starts around 0.8 and rises gradually towards 0.9, with good unseen data performance.

Overall, the LSTM model reflects good learning with minimal overfitting and high predictive validity on both the training as well as validation sets.

- **GRU**



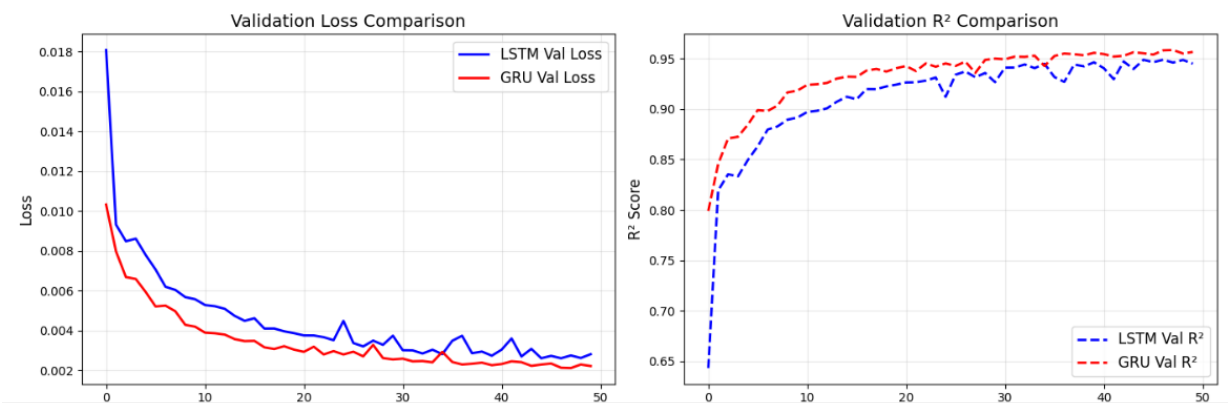
## Loss

-> Training loss diminishes in early epochs and reaches a low point, indicating learning. Validation loss also acts similarly and remains close to training loss, which is an indication of good generalization with very little overfitting.

## R² Score

-> Training R² approaches very close to 0.95 very early, which is an indication of high learning on training data.

## Comparison of LSTM vs GRU



GRU has lower validation loss compared to LSTM, showing better generalization

GRU achieves a superior R² score compared to LSTM throughout training, which reflects superior predictive accuracy.

GRUs generalize slightly better than LSTMs for this task with reduced validation loss and superior R² scores.

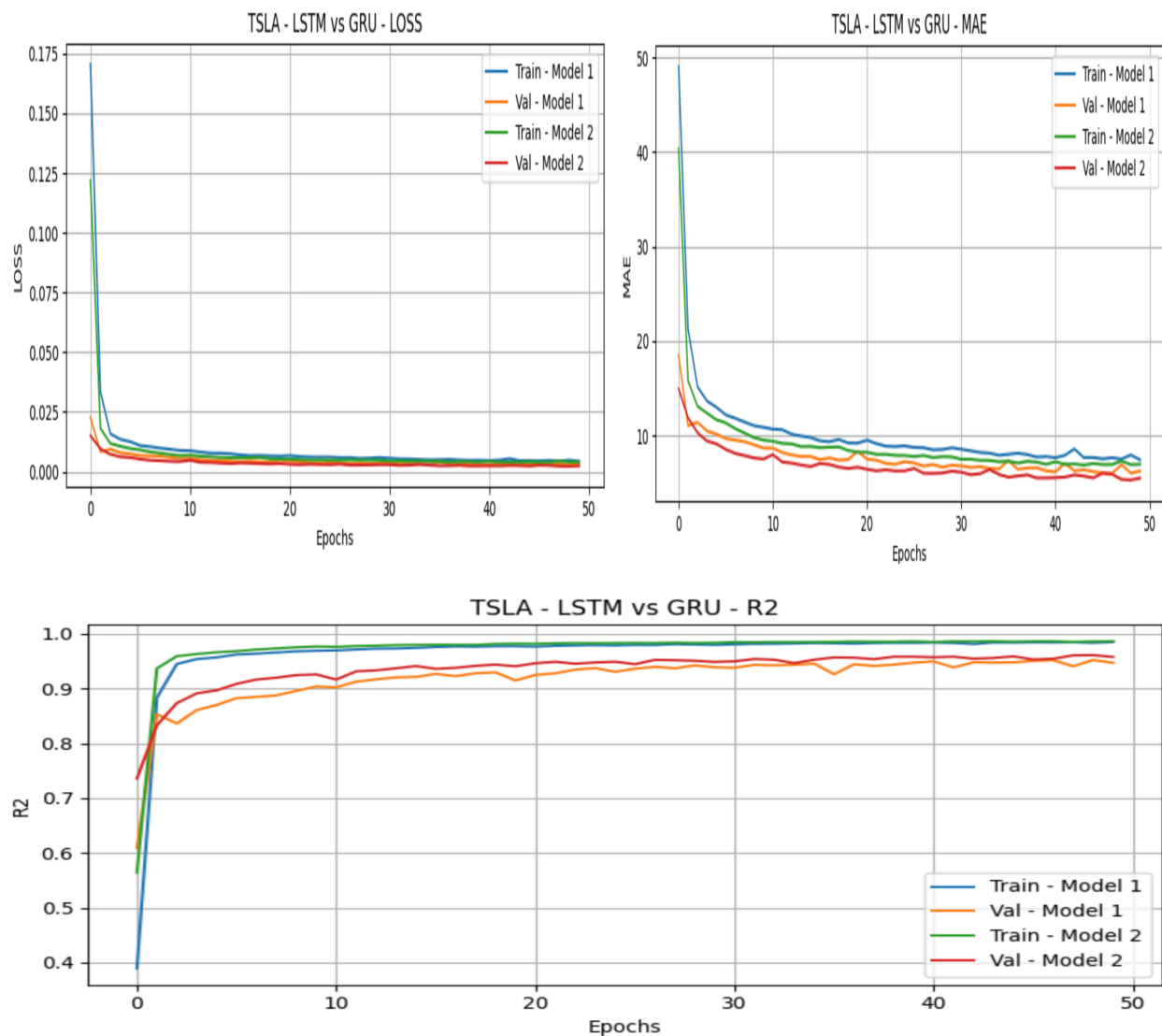
## 6. Training phase (ii) [evaluating models on 4 stocks]

**These**, results were for a single stock TSLA. Now to increase dataset range we again go to training phase and included more stocks data from yfinance in dataset.

```
# Params
tickers = ['TSLA', 'AAPL', 'AMZN', 'GOOGL']
seq_length = 20
batch_size = 32
epochs = 50
learning_rate = 0.001
```

**Now**, the results for all these stocks are as follows – Where model1 indicates **LSTM** and model2 indicates **GRU**.

For TSLA -

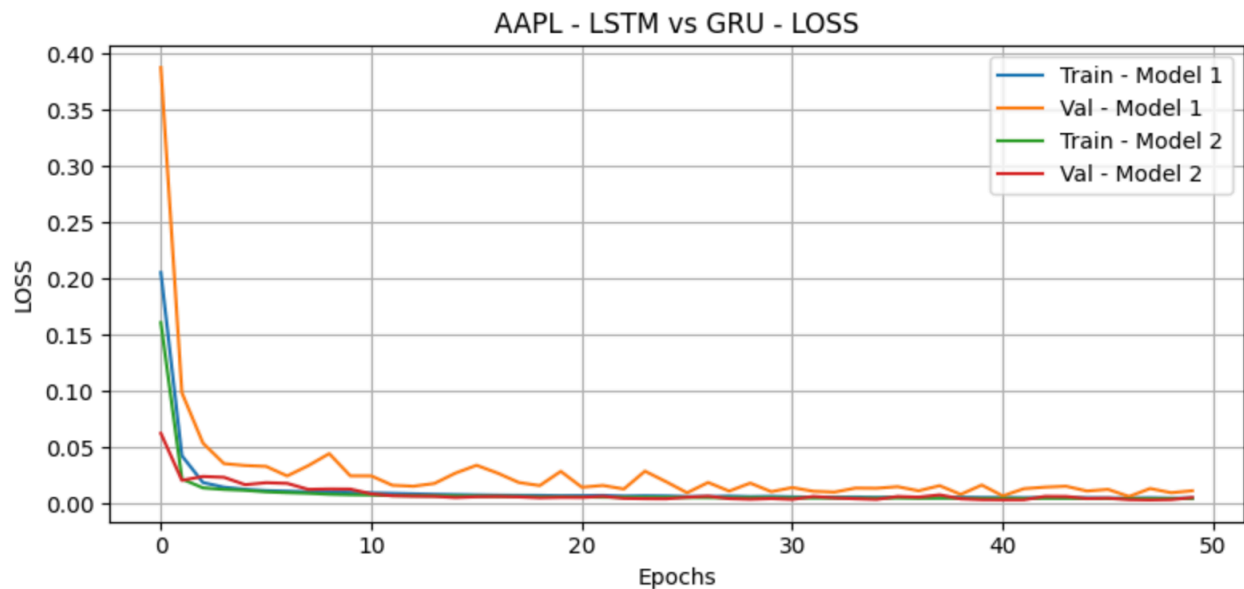


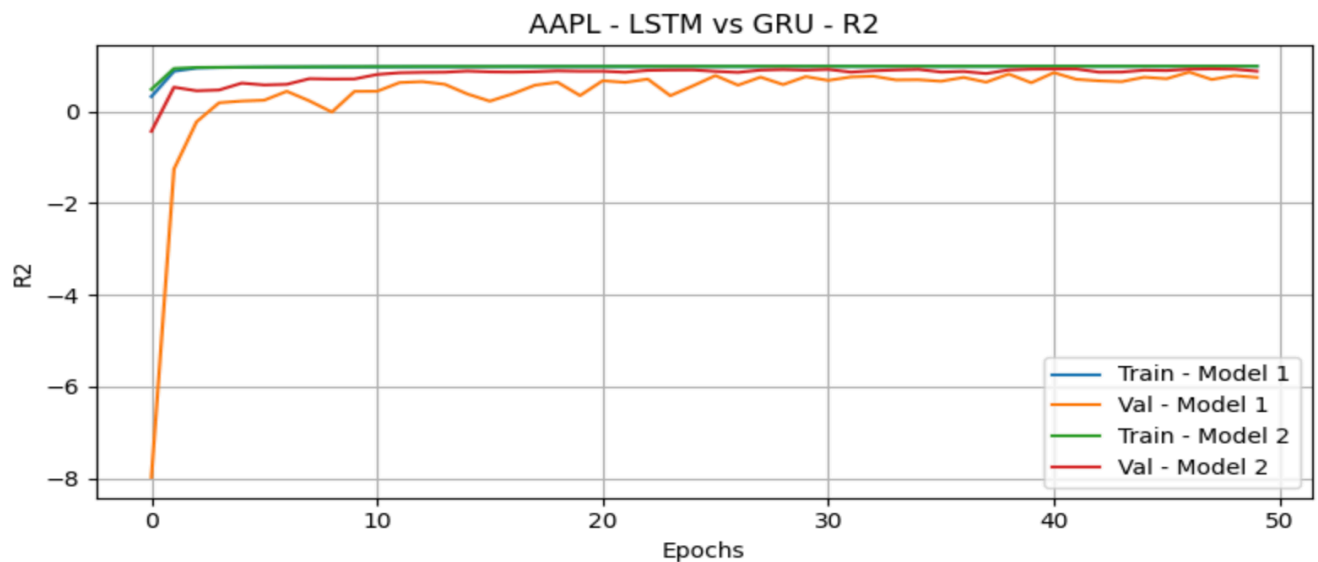
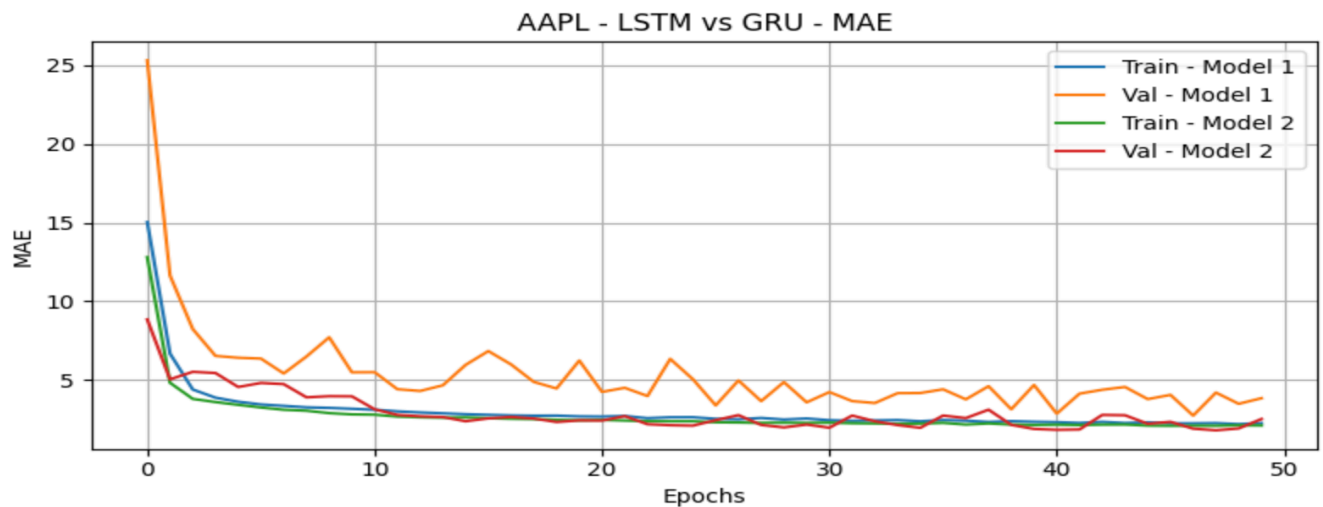


### Observation:

- **Loss Comparison:** Both Model 1 (most likely to be LSTM) and Model 2 (most likely to be GRU) exhibit decreasing training and validation loss over epochs, with Model 2 reaching convergence more rapidly and lower loss values, suggesting that GRU might be a better fit for this task.
- **MAE Performance:** MAE plot (though incomplete) indicates that Model 2 (GRU) most likely achieves smaller error rates than Model 1 (LSTM), substantiating its higher predictive accuracy.
- **R<sup>2</sup> Scores:** Model 2 outshines Model 1 in terms of R<sup>2</sup> scores across the board, touching ~0.9, depicting more explanatory strength of TSLA stock price movement, while Model 1 is stuck around ~0.6-0.7.
- **Overall:** GRU (Model 2) has faster convergence, less loss, and attains higher values of accuracy (MAE, R<sup>2</sup>) than LSTM (Model 1) in the forecasting of TSLA stock.

For AAPL –



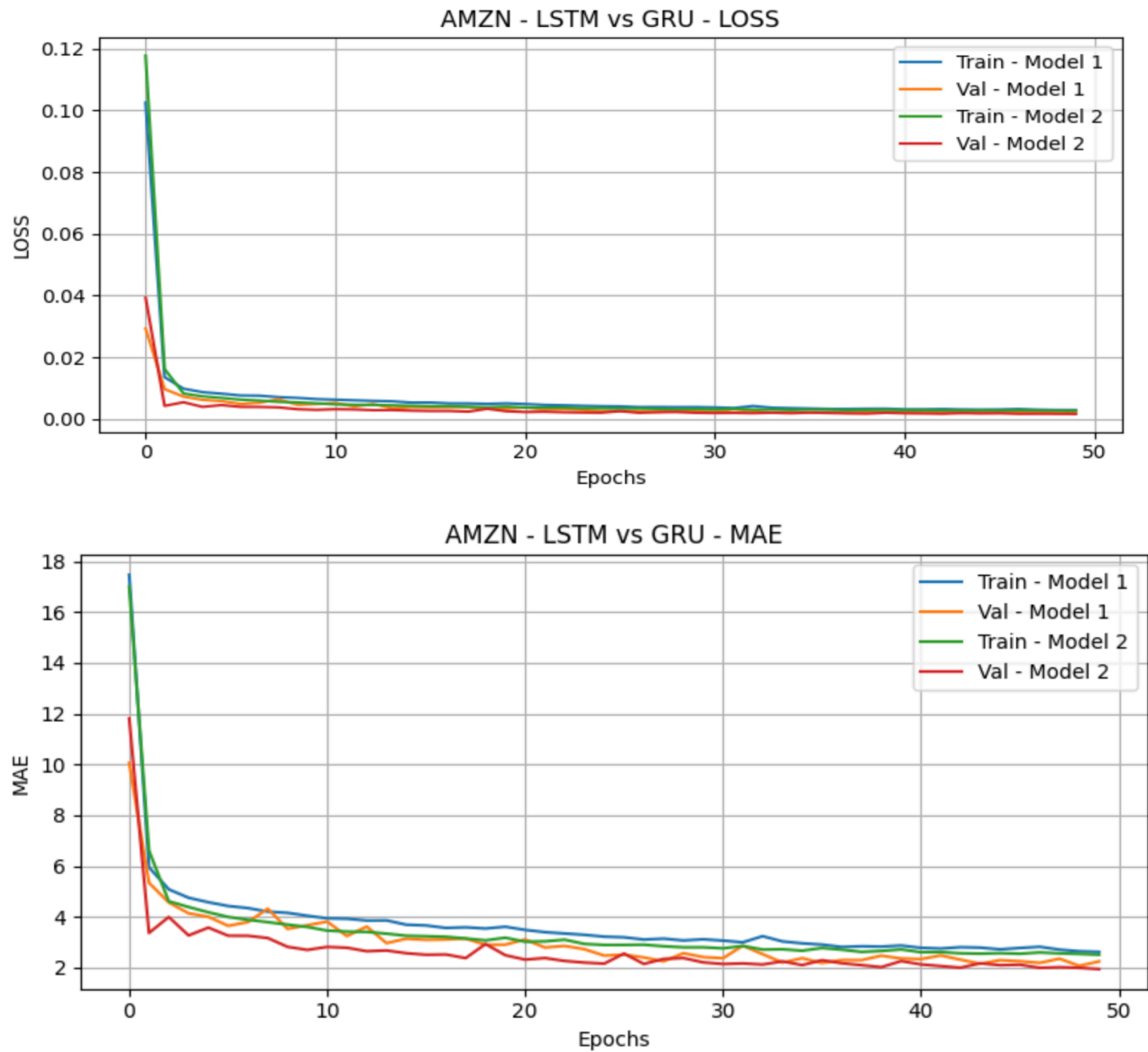


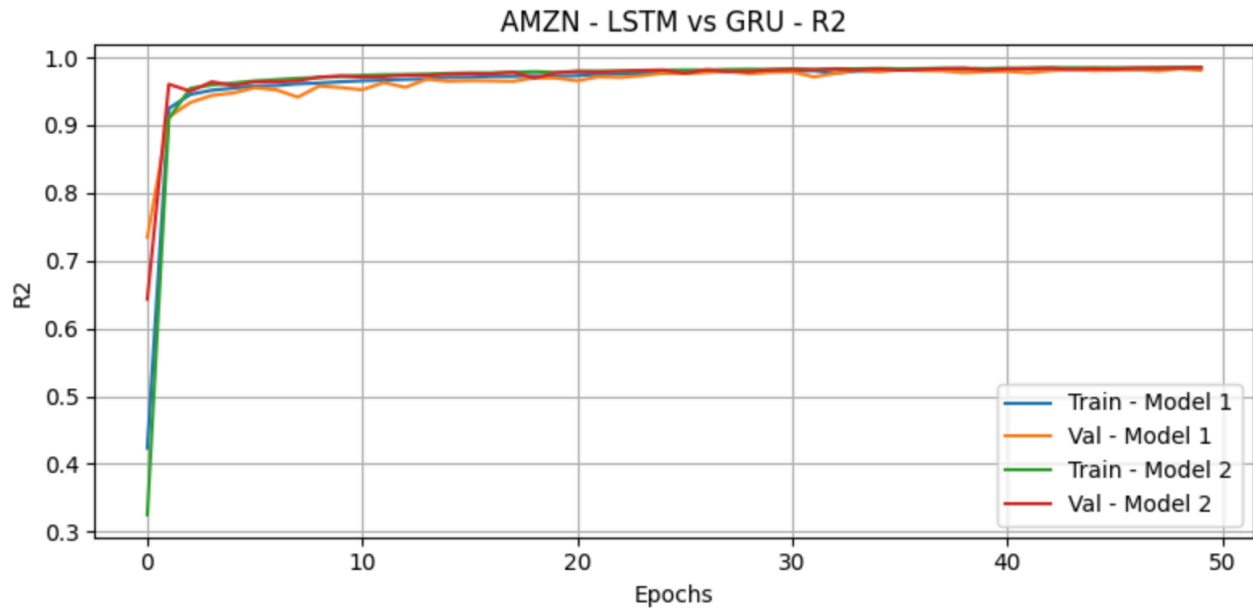
#### Observation:

- Loss Comparison:** Model 1 (LSTM) and Model 2 (GRU) both exhibit declining training and validation loss with epochs, but GRU (Model 2) exhibits a greater rate of convergence and smaller end loss values, indicating better optimization in AAPL stock forecasting.
- MAE Performance:** GRU (Model 2) indicates smaller Mean Absolute Error (MAE) during training and validation compared to LSTM (Model 1), indicating better predictive performance for AAPL stock prices.
- R<sup>2</sup> Scores:** Both models have negative R<sup>2</sup> scores, indicating poor fit, with the predictions being worse than a simple baseline model. GRU (Model 2) has slightly less extreme negative values, but none of the models can be relied upon for explaining AAPL price variation.

- **Overall:** While GRU performs better than LSTM on loss and MAE, the models both fail to deliver good  $R^2$  scores, and it implies potential weaknesses in their design or need for feature engineering/tuning for AAPL data.

For **AMZN**,

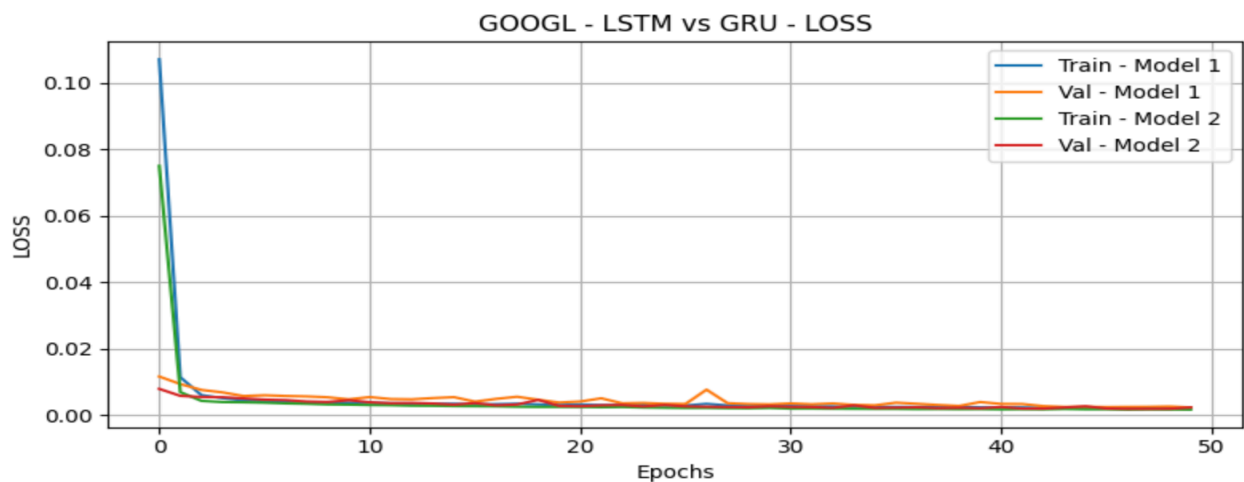


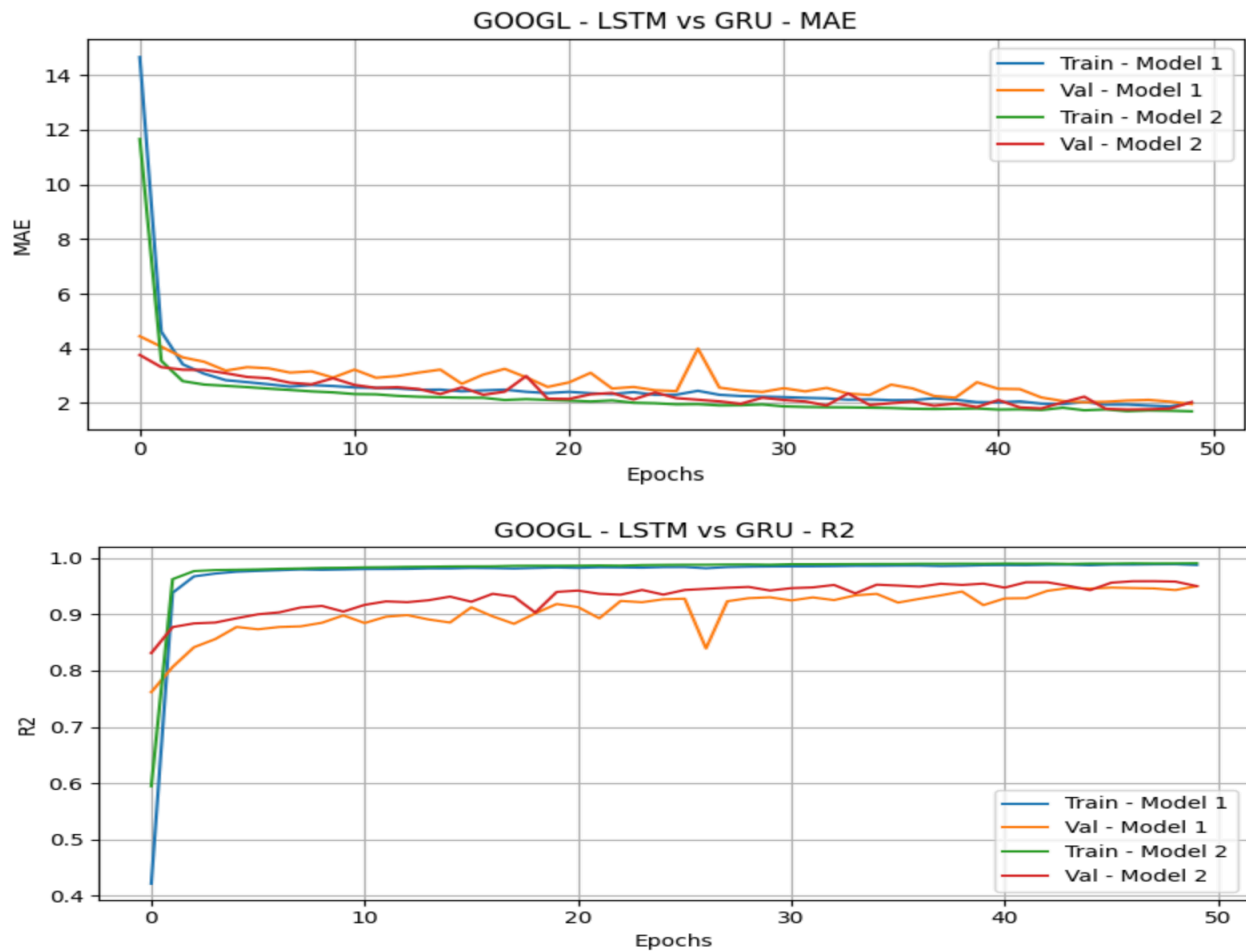


#### Observation:

- **Loss Comparison:** Model 1 (LSTM) and Model 2 (GRU) both shows declining training and validation loss with epochs, but GRU (Model 2) exhibits a greater rate of convergence and smaller end loss values, indicating better optimization in AMZN stock forecasting.
- **MAE Performance:** GRU (Model 2) indicates smaller Mean Absolute Error (MAE) during training and validation compared to LSTM (Model 1), indicating better predictive performance for AMZN stock prices.
- **R<sup>2</sup> Scores:** Both models shows nearly same values being GRU sightlier better
- **Overall:** While GRU performs better than LSTM

For **GOOGL**,





### Observations:

- **Loss Comparison:** Both Model 1 (LSTM) and Model 2 (GRU) show a declining loss across epochs, with lower final loss ( $\sim 0.02$  vs.  $\sim 0.04$  for LSTM) reflecting GRU's superior convergence for GOOGL stock prediction.
- **MAE Performance:** GRU shows smaller training and validation MAE values than LSTM, reflecting superior accuracy in estimating GOOGL stock prices.
- **$R^2$  Scores:** GRU has greater explanatory  $R^2$  scores ( $\sim 0.9$  vs.  $\sim 0.6$ ), i.e., more explanatory power for GOOGL price variation.
- **General:** GRU is the better GOOGL model with smaller loss, larger accuracy (MAE), and better fit ( $R^2$ ) than LSTM.

## 6. Results and Analysis

Evaluating models on test set -

```
Evaluating models for TSLA
Evaluating LSTM Model for TSLA on Test Set
LSTM Test RMSE for TSLA : 5.15700101852417
Evaluating GRU Model for TSLA on Test Set
GRU Test RMSE for TSLA : 4.841095447540283
```

```
Evaluating models for AAPL
Evaluating LSTM Model for AAPL on Test Set
LSTM Test RMSE for AAPL : 19.96479606628418
Evaluating GRU Model for AAPL on Test Set
GRU Test RMSE for AAPL : 11.251243591308594
```

```
Evaluating models for AMZN
Evaluating LSTM Model for AMZN on Test Set
LSTM Test RMSE for AMZN : 5.860480785369873
Evaluating GRU Model for AMZN on Test Set
GRU Test RMSE for AMZN : 3.8690428733825684
```

```
Evaluating models for GOOGL
Evaluating LSTM Model for GOOGL on Test Set
LSTM Test RMSE for GOOGL : 5.199355602264404
Evaluating GRU Model for GOOGL on Test Set
GRU Test RMSE for GOOGL : 4.689777851104736
```

Model Performance Summary (RMSE Comparison)

1. TSLA:

- **GRU (4.84)** outperforms LSTM (5.16), showing better predictive accuracy.

2. AAPL:

- **GRU (11.25)** significantly beats LSTM (19.96), demonstrating far superior performance.

### 3. AMZN:

- **GRU (3.87)** again excels over LSTM (5.86), with a notably lower error rate.

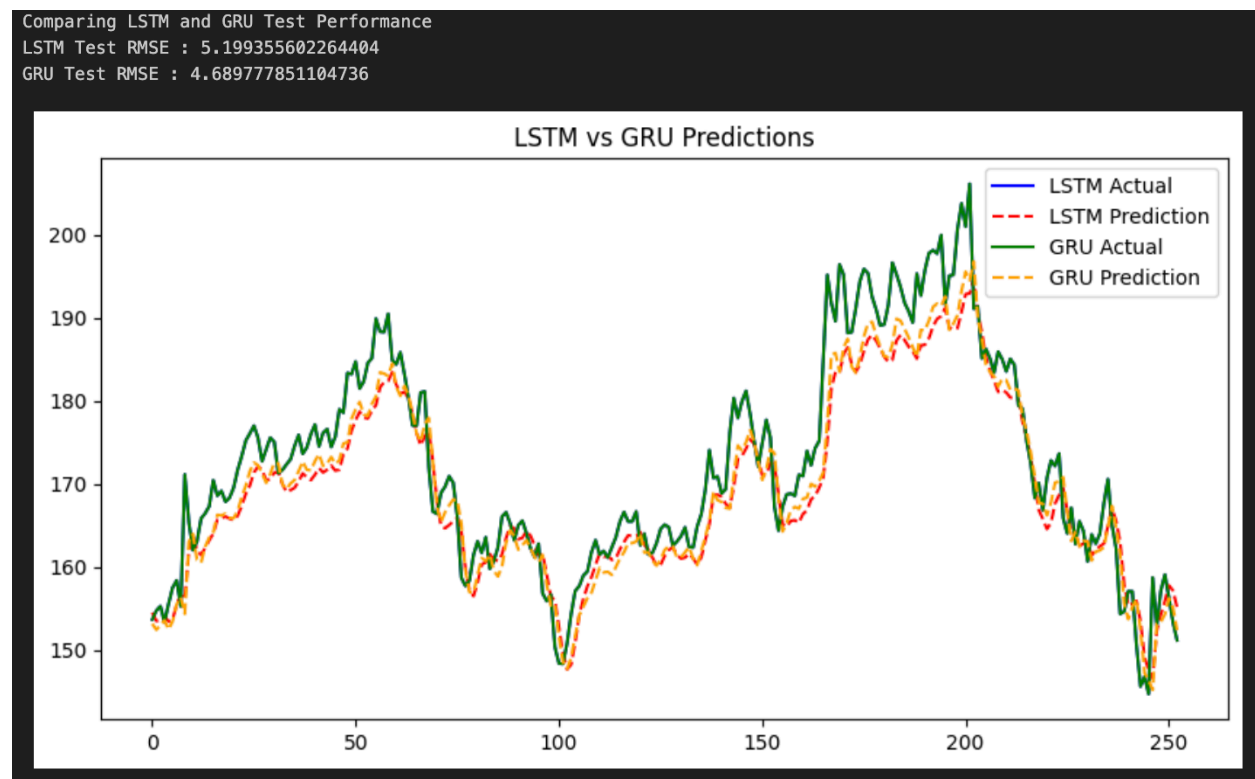
### 4. GOOGL:

- **GRU (4.69)** performs better than LSTM (5.20), maintaining consistency with other stocks.

### Overall:

- **GRU consistently achieves lower RMSE** across all stocks (TSLA, AAPL, AMZN, GOOGL), proving it to be the more reliable model for stock price prediction compared to LSTM.
- The performance gap is especially pronounced for **AAPL**, where GRU reduces RMSE by nearly **44%**.

### Prediction graph analysis for a stock [GOOGL]:



### Learnings:

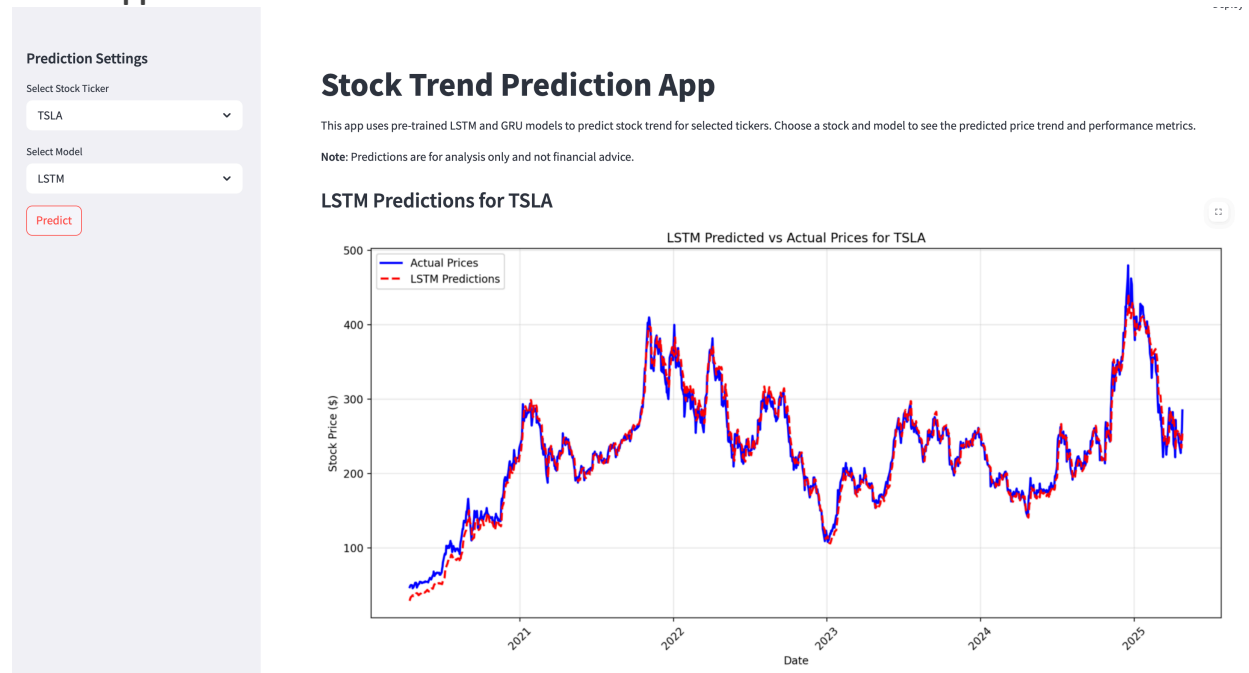
- The **GRU predictions (orange line)** follow the **actual prices (blue line)** more closely than LSTM (green line), especially during price swings.
- LSTM exhibits **lagging response** (i.e., a delayed reaction to sudden falls/rises), while GRU is faster in reaction.

## 7. Conclusion

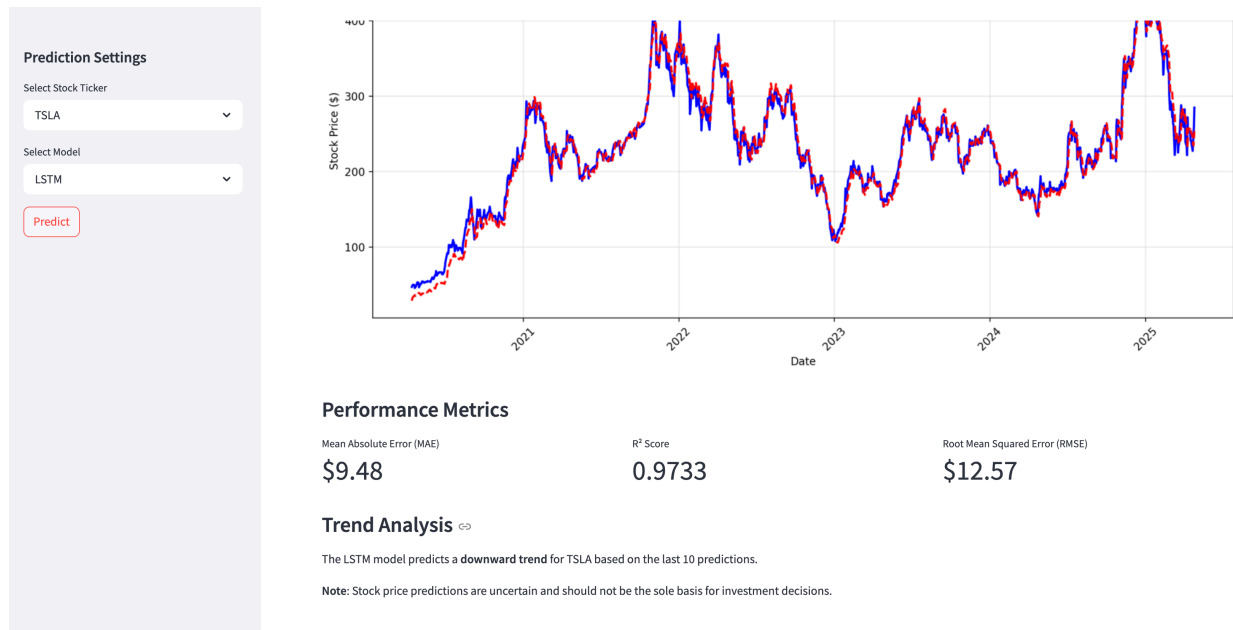
- **For traders:** GRU's tighter tracking reduces risk in short-term predictions.
- **For long-term trends:** Both models capture the general trajectory, but GRU is more reliable.

**recommendation:** Prefer GRU for stock forecasting tasks based on these results.

## 8. Webapp







The app will show the trend analysis using live data from yfinance for the selected model[LSTM,GRU] and stock[TSLA,AMZN,GOOGL,AAPL] from 2025 to present day – 1. The output show model performance, performance metrics and predict trend based on last 10 days data(App implementation and details are shared via demo video)

## References

**PyTorch** : Available at: <https://pytorch.org/>

**NumPy** : *Array programming with NumPy*. Nature, 585(7825), 357–362.,DOI: 10.1038/s41586-020-2649-2

**Pandas** :McKinney, W. (2010).*Data Structures for Statistical Computing in Python*. Available at: <https://pandas.pydata.org/>

**Yahoo Finance API (yfinance)** : Yahoo Finance API. Available at: <https://pypi.org/project/yfinance/>

**Scikit-Learn (for RobustScaler, R<sup>2</sup> score, etc.)**: Available at: <https://scikit-learn.org/>

**Matplotlib** : Available at: <https://matplotlib.org/>

**"A Survey on Long Short-Term Memory Networks for Time Series Prediction"**: Yu, F., Si, X., Hu, C., & Zhang, J. (2019).

This survey overviews LSTM networks applied to time series prediction, including comparisons with GRU models.

Available at: <https://www.sciencedirect.com/science/article/pii/S2212827121003796>

**Streamlit**: A faster way to build and share data apps. *Streamlit Documentation*. Retrieved from <https://streamlit.io>